ГЛУБОКИЕ ПОГРУЖЕНИЯ

Как меняется процесс разработки программного обеспечения в Anthropic

Подробный рассказ о том, что изменилось в подходе к разработке программ обеспечения в ведущей лаборатории искусственного интеллекта. Все больш проверяется и тестируется с помощью ИИ, «команды двух пицц» по-прежне деле и многое другое. Подробности из жизни Anthropic

ГЕРГЕЙ ОРОШ

28 ИЮЛЯ 2026 ГОДА ОПЛАЧЕНО

Значительно усовершенствованные инструменты на основе искусственно интеллекта меняют подход к разработке программного обеспечения, и я хо взглянуть на то, как может измениться будущее разработки программного обеспечения под их влиянием. Для этого лучше всего подойдут команды, которые больше всего «подсели» на искусственный интеллект, — сами лаборатории ИИ.

Итак, я посетил две ведущие лаборатории в области искусственного интеллекта, чтобы посмотреть, как команды и инженеры работают кажды день. В этой статье и в следующей я расскажу о том, как искусственный интеллект меняет привычные для многих из нас принципы разработки программного обеспечения, а также о том, что осталось прежним, несмотр распространение ИИ.

В следующей статье мы сравним результаты исследований Anthropic и Op чтобы понять, как их методы работы могут повлиять на развитие разработ программного обеспечения в целом.

Внутри штаб-квартиры Anthropic (слева). На стене — вехи развития искусственного интеллекта (справа)

Спасибо Anthropic за то, что показали мне свою лабораторию в Сан-Франц
Я поговорил с четырьмя сотрудниками:

- **Кейтлин Лесс**, руководитель инженерного отдела платформы Claude, ч организация владеет инфраструктурой, на которой работает Claude
- **Джарред Самнер**, создатель Bun, ныне работающий в Anthropic над Bu Claude Code
- **Тарик Шихипар**, занимающийся разработкой и обучением в рамках Cla Code
- **Дэвид Херши**, сотрудник подразделения Anthropic Applied AI, выполняк функции инженера по продажам и работающий с такими клиентами, ка Cursor, Cognition и Perplexity

Благодаря им я понял, в каком направлении движется ведущая лаборатор разработке искусственного интеллекта — и, возможно, вся отрасль в цело

Прежде чем мы продолжим, сообщаем, что The Pragmatic Engineer будет на л каникулах в течение следующих полутора недель. Это значит, что на этой не

Возвращаясь к сегодняшнему подробному разбору, отметим:

1. **Сложный и долгий процесс: управляемые агенты Claude**. Один из сам сложных проектов занял у команды платформы Claude шесть месяцев позволил создать новый примитив для использования на уровне инфраструктуры агентов. Инфраструктурные проекты все еще нуждаю перестройке архитектуры в процессе разработки, и на то, чтобы довес до ума, требуется время.
2. **Проект, рассчитанный на 12 месяцев, был завершен за 11 дней: перен Bun на Rust.** Раньше на перенос проекта из 500 тысяч строк на другой у небольшой команды уходил год, что делало его нецелесообразным. Благодаря Fable и 165 тысячам токенов на это у создателя проекта ушл меньше двух недель.
3. **Изменение инженерных практик**. В лаборатории искусственного интеллекта, где работают более 3500 сотрудников, процесс создания прототипов стал более гибким, верификация занимает больше времен чем внедрение, а проверка кода и тестирование все чаще выполняютс помощью ИИ.
4. **Изменения на уровне команд.** Дизайн становится более непрерывным менее формальным, команды работают над большим количеством проектов, в каждом проекте задействовано не более двух инженеров и
5. **Все по-прежнему:** команды из двух человек, важность планирования, актуальны для сложных проектов, переключение контекста — непрост задача, соотношение времени, затрачиваемого на кодирование и тестирование, меняется не так сильно.
6. **Стоит ли менять архетип «выдающегося» инженера-программиста?** Ценятся глубокое понимание, в том числе того, что происходит на уров ниже того, над чем вы работаете, а также способность координироват работу.

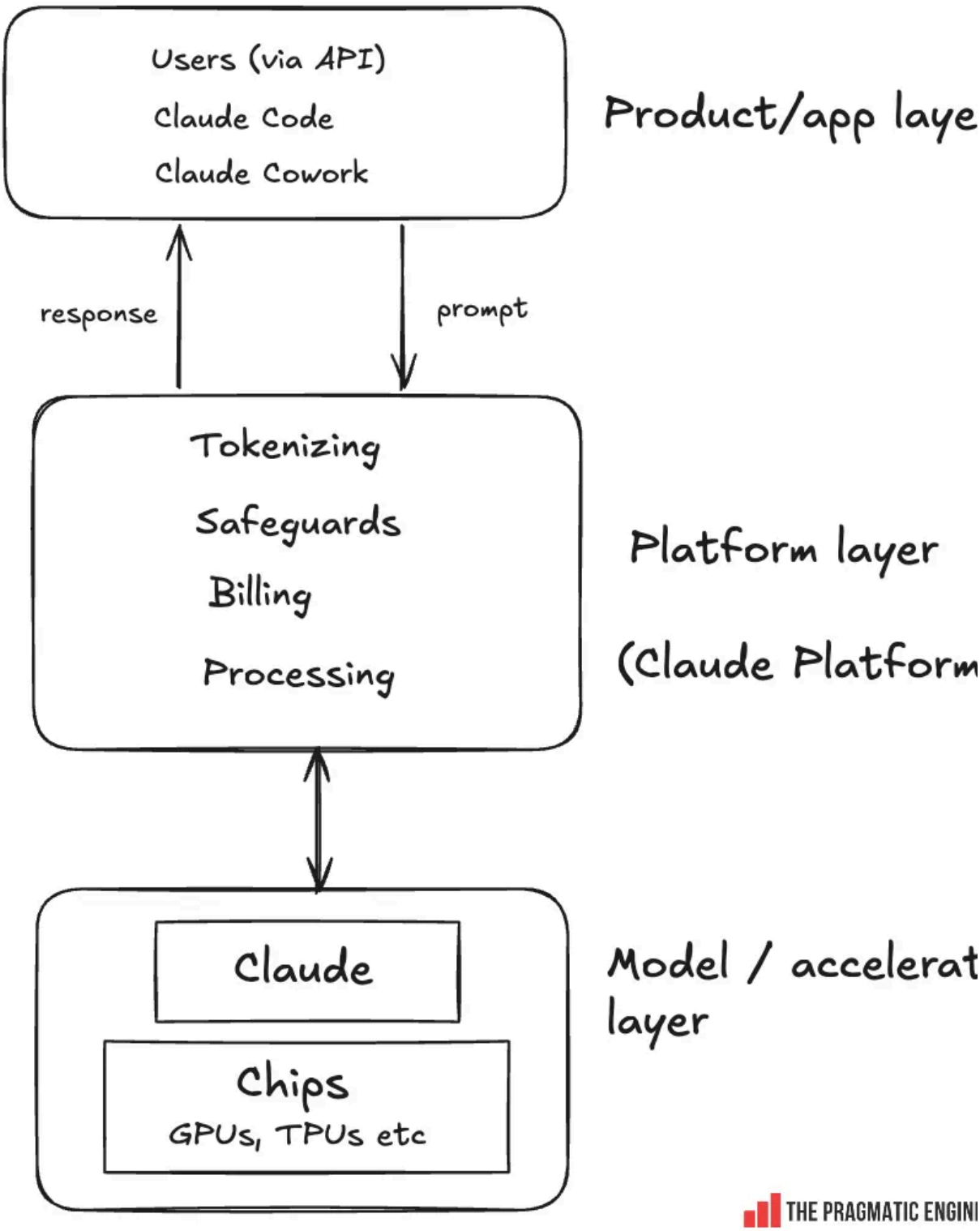
работают с ИИ в лаборатории, тем меньше они боятся, что их работа исчезнет.

1. Complex & long: Claude Managed Agents

The Claude Platform team's most complex project in the past year was building Claude Managed Agents, a pre-built harness for production agents that runs in th cloud on infrastructure managed by Anthropic, or on your team's own infrastructu with any sandbox you choose. The project took around six months from idea unt launch in April. Katelyn Lesse, head of engineering for Claude Platform, shared th story.

With Katelyn Lesse, at Anthropic

This team sits between the model/accelerator layer (Claude models operate on C and the product/application layer (with products like Claude Code and Claude Cowork):



Where Claude Platform sits inside of Anthropic

> "We're on the 'token hot path.' The prompt comes in, then we tokenize it. Then things like safeguards and billing all happen within our layer."

What the Claude team calls "Platform," I think more of as "API." Claude Platform operates the API, and owns responsibilities an API would have. *Of course, the tea does more than that, and Claude Managed Agents is one case we cover here.*

The platform layer is being migrated from Python to Rust. Originally, this layer w written for Python for the "usual" reasons at AI companies: it's a convenient lang and AI researchers use Python already, which enables quick iteration. But Pythor single-threaded, and at scale, when the API is under high load, it's not as perform as Rust.

Harness infrastructure demand

The project came together due to customers wanting their own "harness infrastructure", says Katelyn:

> "We started with a model where you get an API to define an agent, then you ge API to start a session with an agent. The reality of what the world wants and r right now is people running their own infrastructure. So, we started to build a s hosted sandbox.
>
> But then, what we started to hear from lots of customers is that they're trying hack harnesses together, running their own "harness infrastructure," and this g us the idea for Claude Managed Agents."

The largest part: planning

In this project, Katelyn said the single biggest matter was planning:

> "There are products you can jump straight to prototyping, but then there are o where you need to start by architecting it properly. For example, if we build a TypeScript CLI – which is pretty trivial for what needs to be built – we could g

> Of course, we did some upfront prototyping for Managed Agents: hacking and spiking things. But prototyping itself was more about understanding the requirements.
>
> **Our planning process looked more like a typical pre-AI planning process.** You know how every team has the project, where everyone comes up with some ve of the same idea and people keep floating and circling it around until you final it? Managed Agents was this for our team. When we started the project, we ha documents dating back up to two years about ideas and suggestions.

Post-planning, when the project officially kicked off, a PRD (product requirements document) was created:

> "In the end, it was the Product Manager and the Tech Lead on our API Agents who decided to pull the trigger and kick off this project. We'd get in a room, go through it, and get aligned. But it wasn't just us: we'd have to align with teams around the business, other cloud providers, and other engineering teams. For example, we have a sandboxing team inside of the Platform org: and so this te was consulted on the design of Managed Agents, given this product would sp lot of sandboxes.
>
> **Just like before, we had a PRD, it was a Google Doc.** We used a Google Doc because we needed to coordinate all interested people. This has not gone aw
>
> Similarly, my product counterpart and I run product reviews."

Some processes from before AI, like the PRD, are still useful in complex projects for getting large groups of people on the same page.

Build for an internal customer first

code with an agent harness was a similarly shaped problem to the one they wan
solve for customers. The thinking was to start by solving it for the Claude Code t
before tackling it in a more generic way for customers.

Internal teams are more fluid than before AI. Katelyn:

> "Pre-AI, we might have hit the Claude Code team up with a bunch of big requirements documents, and they would have then hit us back with another s documents. Now it was much easier: someone on our team built a few components, took it over to the Claude Code team, and they started to hack a it. We could figure out how this component plugs into this part of their produc the other way around. It was just a faster and easier process, getting this first internal version of the product up and running.
>
> Aligning with other teams on interfaces remains important, and it's easier. Ba the day, you'd have to come with a fully spec'd interface to use. Now, we coulc a lot more fluidly: we could stand up a stub service that shadowed traffic to st with, and iron out the interfaces with the Claude Code team as we went. They some hacking on it and gave feedback, we made changes while building out t service under the interface, then went back to make it work."

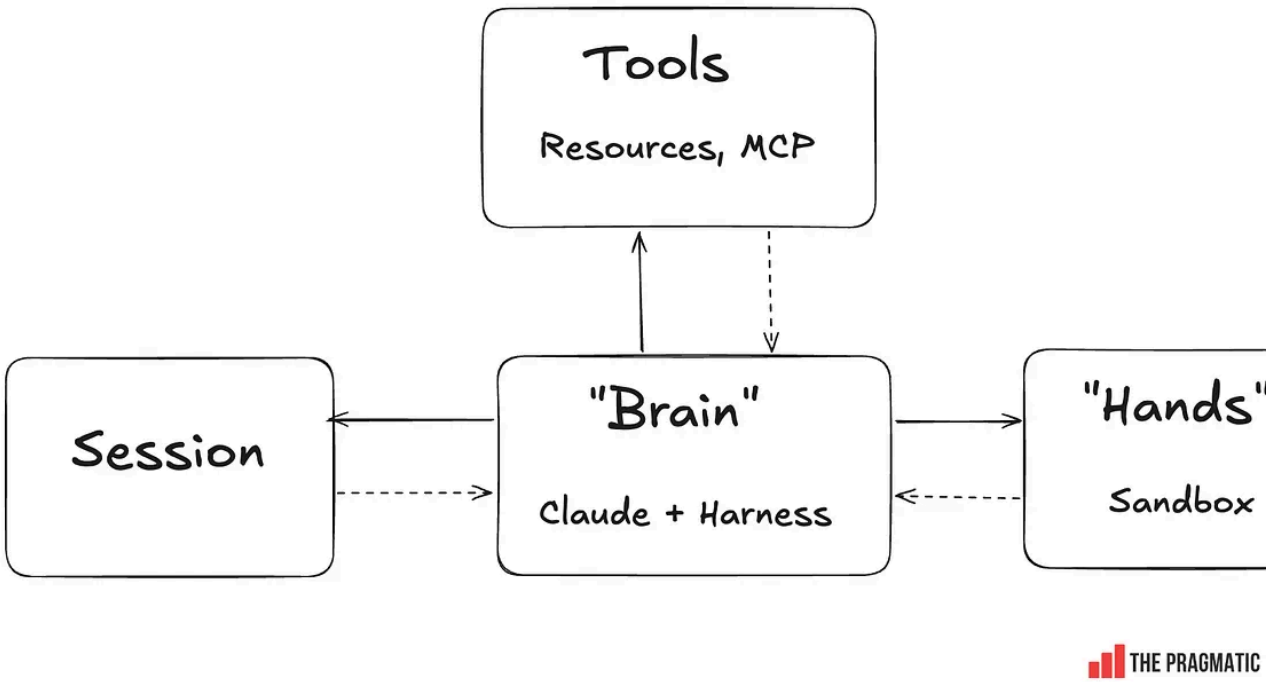
They launched a service for Claude Code's mobile app to spin up a sandbox, boo
Claude Code and run it. The service went to production and the Claude Platform
took the learnings.

Re-architecting midway through

It's likely a familiar scenario many engineers can relate to, that after planning a p
and getting underway, you see that you're going to need to change the architectu
happened on this project, too.

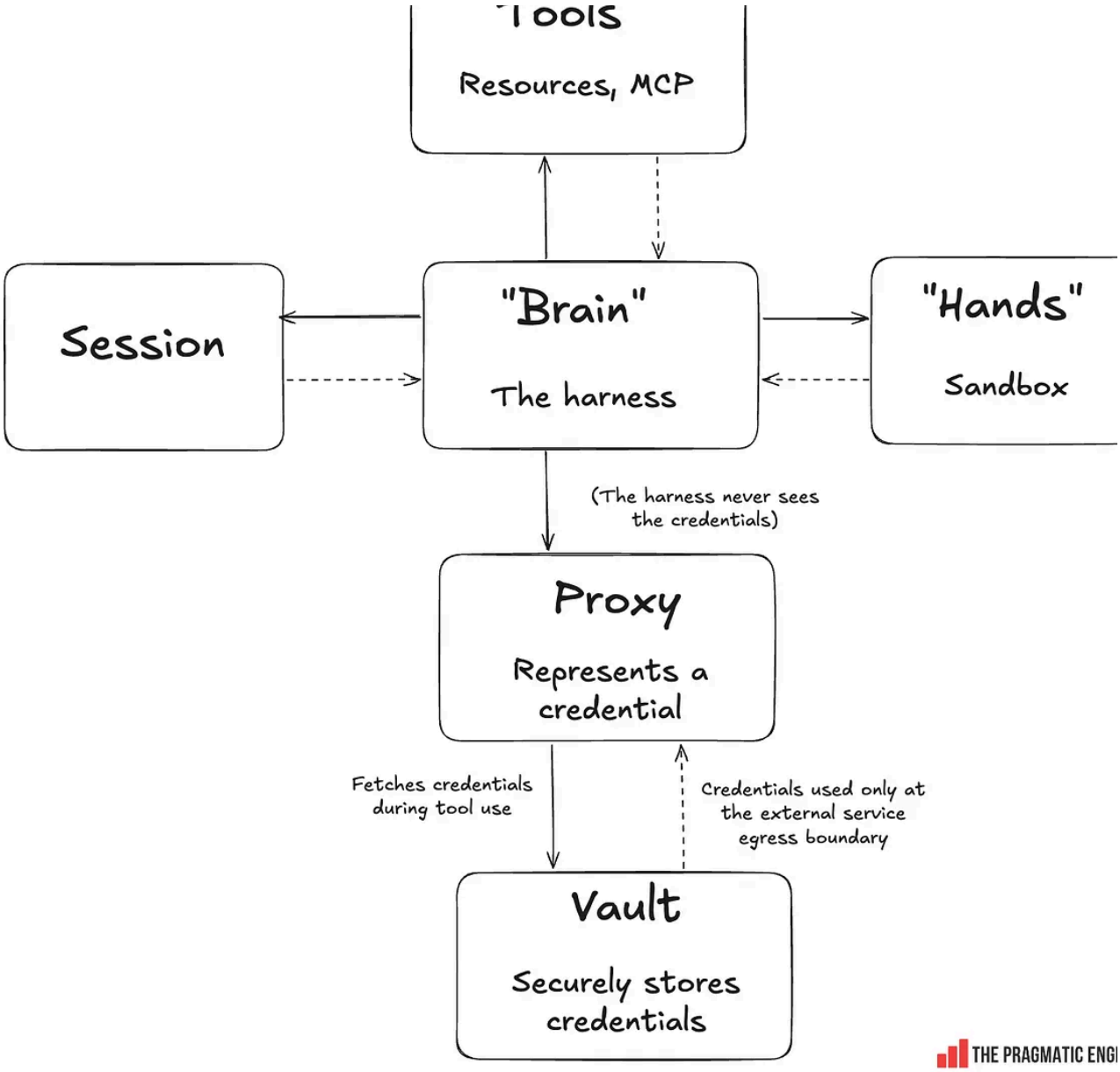
**The platform team ended up re-architecting Managed Agents based on learning
from the Claude Code "spike."** Re-architecting meant decoupling the "brain" of C

about each other.



High-level architecture of Claude Managed Agents after the re-architecture

The team also built an abstraction around vaults and credentials. Credentials ca safely be stored inside a vault. All calls using credentials are made via a proxy w has a session token. It is the proxy that fetches the right credentials from the va credentials are never seen by the agent, sandbox, or session. Credentials are onl injected at the egress boundary when the service is invoked:



Adding credentials the harness never sees

Internal “dogfooding” helped surface hard problems to solve. A few examples:

- Reliability and scalability: these are really hard to do well for agents because connection to the sandbox is lost, the whole agent dies and you lose state
- Credentials and access control: also hard and problematic, especially when building the service

The Managed Agents team shared [more about this re-architecting project](#).

Managed Agents is one of the biggest projects the Claude Platform team has bu
and more complex than it looks: for example, adding support for running agents
AWS, GCP and Azure.

2. Twelve-month project done in 11 days: Bun rewrite to Rust

As covered before, Jarred Sumner is the creator of Bun, a popular JavaScript run
with 22 million monthly downloads currently and Claude Code as a dependency.

With Jarred Sumner (left), creator of Bun

also-performant, memory-safe language like Rust could be an option – except th
rewrites like this turned out as follies in the past. Jarred (emphasis mine:)

> "Historically, rewrites are a terrible idea. Excluding comments, Bun is 535,496
> of Zig. A rewrite in another language would take a small team of engineers a f
> year. It would mean freezing bugfixes, security fixes or feature development fo
> time. The least risky approach to getting something shippable would be a
> mechanical port from Zig to Rust, with the minimal number of behavioral char
> using the exact same test suite we already use for testing Bun.
>
> Fortunately, Bun's own test suite is written in TypeScript which means it doesn
> depend on the runtime's programming language.
>
> **A year of zero user-facing impact was not an option we could consider.** So,
> enforcement through code style to fix stability issues was our best bet, and w
> plan when we added Rust-inspired smart pointers to Bun's codebase.
>
> But honestly, I didn't want to do it. Homegrown smart pointers offer worse
> ergonomics than Rust, with none of the guarantees."

But then, Jarred asked if AI could do the heavy lifting and wondered how much th
migration could be sped up. **In the end, he completed the rewrite from start to m
in 11 days, using 64 parallel agents and $165,000 in tokens at API price.** Here's
Jarred on how his AI-heavy rewrite compared:

> "By hand, I think this would've taken three engineers with full context on the
> codebase about a year, during which time we wouldn't be able to improve Noc
> compatibility, fix bugs, fix security issues or implement new features. We neve
> would've done that. The realistic alternative was to do nothing and keep fixing
> bugs at the top of this post forever."

There was a lot more to the project than typing out the "...make zero mistakes"
prompt:

but worked on different files in the same codebase

- He created an orchestration system where each AI agent came up with suggestions of what to change, but did not make a change to the file to avoi conflicts; an orchestrator AI agent created the commits
- The most time and tokens went on fixing the compile bugs, tests, and verifyi that things worked
- Bun itself has a very robust test harness: when all tests pass, it's a high-confidence signal that the rewrite works
- Crucially, Jarred is *the* ultimate domain expert in Bun: he created the project knows the codebase better than anyone

The rewrite has been shipped to production and powers Claude Code today.

We cover a lot more on this in [What can we learn from Bun's rapid Rust rewrite with

3. Changing engineering practices

So, what has changed in how teams build software at Anthropic, compared to th AI days? That's the question of this article, and it seems that many things are dif Let's go through it:

AI lab-specific practices

Some things as normal as breathing at AI labs like Anthropic stand out as differe with an outside perspective:

- **Everyone runs multiple AI agents all the time.** Running 3-10 parallel agents i given. Folks I talked with had their agents running in the background or clou
- **No token budget, usage not tracked.** One major difference between AI labs a everyone else is that there *really* is no token limit or token leaderboards that promote tokenmaxxing; people already use agents all the time.

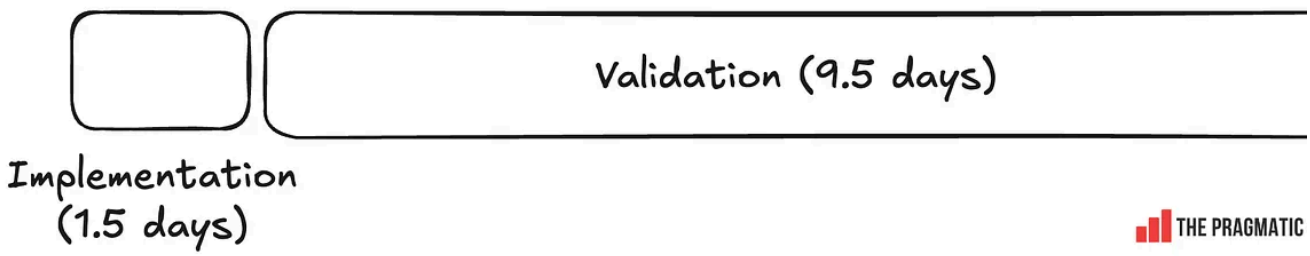
has unlimited tokens, it's pretty easy to prototype any idea.

Prototyping and "spiking" is far more fluid

It was several times faster to prototype early approaches for Claude Managed Ag
Similarly, "spiking" the Claude Code mobile backend implementation was much f
than pre-AI, Katelyn told me.

Verification takes longer than implementation

Jarred made a point about the split between implementation and validation in his
day rewrite to Rust. Roughly, it was:



Implementation of the Rust rewrite took far less time than fixing it up, then validating that it works as expected

The "implementation" part of rewriting the code from Zig to Rust took about 15%
the time, while 85% went on fixing things up: getting it to compile, fixing tests, ve
that it worked.

Most tokens no longer spent on implementation

Thariq:

> "We see that few tokens are spent on actual implementation. Most are spent
> discovery of unknowns, prototyping, mocking, and then in verification and test

Code review and more testing by AI

Jarred:

> "Critiquing the code and testing it with agents is a new approach we do a lot r
> of. I think a lot about trust when you merge a lot of code. How do you merge 1
> PRs a day, and make sure the code works? At this pace, you need to trust the
> without the ability to read it all yourself. And I think it's a few things:
>
> - **Code review**: it needs to be really good and automated. I'm clearly tooting
> own horn here, but I find Claude's code review to be really good. Claude's
> review catches bugs that would take me an hour of closely reading the cc
> figure out. The caveat is that it's expensive!
> - **Security scanning**: for this Rust rewrite we did 11 runs of the Claude Secu
> Scanner.
> - **Fuzz testing**: we've also been doing different types of fuzzing (fuzz testin
> where we had Claude write a fuzzer for things like parser fuzzing.
>
> Running out-of-process testing, where it happens in a different process/sessi
> from coding, is one way to build trust in the code. I expect more of this."

New pattern: fanning out work to AI

Jarred described a new way he works:

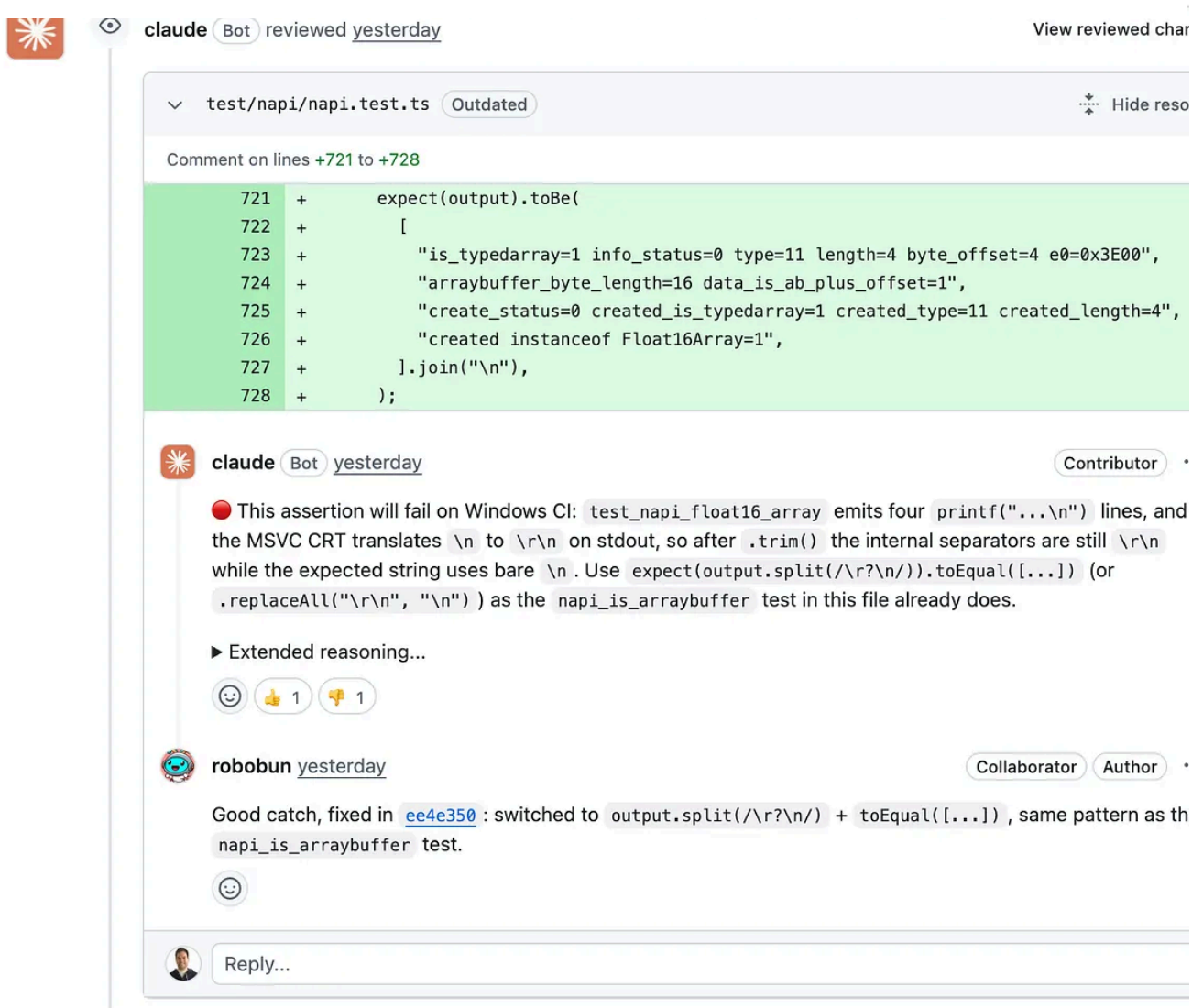
> "A new approach I'm using is fanning out a lot of the work to many Claudes at
> same time. I did this with the Bun rewrite, but I use it for other work. This appr
> works very well for me, and I feel it's pretty underused."

Time-saving automations powered by agents more widespread

- Every time someone files an issue, Claude runs to try and reproduce the issu succeeds, it starts another container, which then tries to fix the issue and su PR.
- The agent tasked with submitting a PR has to write a test that fails in the sys version (the one without the patch) of Bun, and passes in the debug build wi patch, before it is allowed to submit a PR
- There are other automations, like if there is no test, the PR is auto-rejected; a linters are run: Claude Code review is run, CodeRabbit's code review is run, a agents go back and forth on the GitHub pull request

Auto-merge of pull requests: coming soon?

Pull requests are merged manually when all quality gates pass, but this could be automatic at some point. Once all the above checks pass, all (AI) code review comments are addressed, tests are added to new code, etc. As an interesting as lot of GitHub activity is Claude talking to Claude!



Claude talking to Claude. Source: Bun

But manual merging may vanish in low-risk cases, at least for the Bun project. Ja told me:

> "Today, a person presses 'merge' but within a few months, I expect:

- Automated reviewer LGTMs
- → another Claude with a fresh context window judges if it's simple and low l radius
- → if it is: auto-merge!"

Test assumptions with each model generation

> “The thing with agents is that you have to revisit any assumptions you have m because it can change with a new model generation. For that reason, we delet 80% of the Claude Code system prompt recently because the model has gotte smarter.
>
> **Using HTML is another assumption we needed to re-examine.** HTML is one c those things which Claude is a lot smarter at than many of us expected. I’ve s preferring HTML as an output format over Markdown, and see this being usec others on the Claude Code team.
>
> HTML can convey much richer information compared to markdown, HTML documents are easier to read and share.”

4. Team-level changes

At Anthropic, there are also changes in how engineering teams operate, compare pre-AI.